AI SYSTEMS ARCHITECT

Complete Roadmap

From Wrapper Dev to Unfireable

(2026 Edition)

Himanshu Ramchandani

Microsoft - MVP

Wrapper Dev V/S AI Architect

The 2023 Engineer (Wrapper)

- ❌ Writes prompts for OpenAI
- ❌ Pays $1,000/mo in API costs
- ❌ Breaks when internet goes down
- ❌ **Market Value** $0 - $80k (Replacing by No-Code)

The 2026 Architect (Builder)

- ✅ Fine-tunes local models (SLMs).
- ✅ Orchestrates multi-agent loops.
- ✅ Builds private, offline RAG.
- ✅ **Market Value** $150k - $300k (High Demand)

Mission

In this roadmap, you will not learn topics. You will build **6 Production Artifacts**. This is your new portfolio.

Table of Contents

How Each Artifact Is Structured

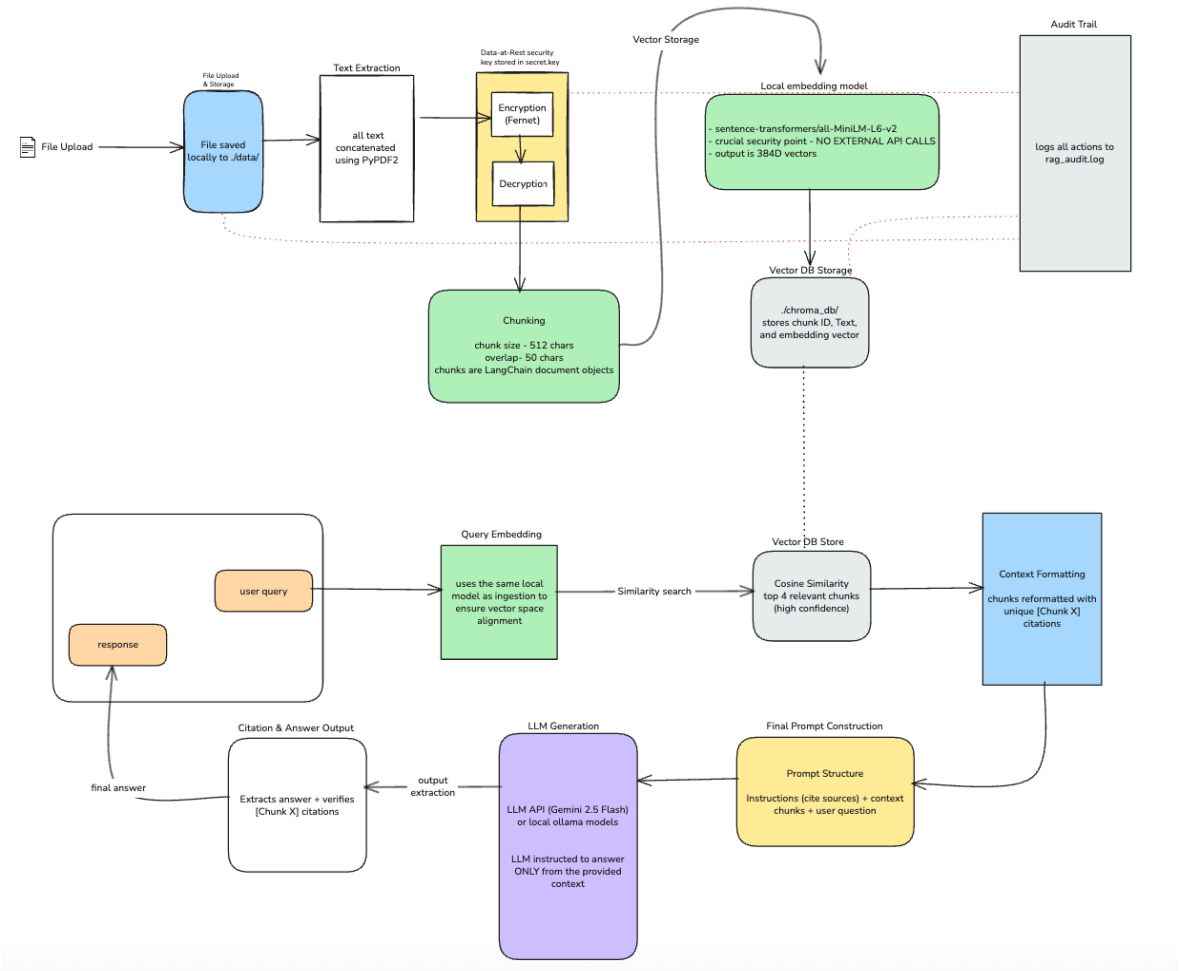
Every artifact follows the same pattern:

- Problem we are solving
- What you actually learn (Contextual Curriculum) - Think of this as your mini roadmap for each project. You only learn what you need to build the solution.
- Tech stack
- Proof of work

Let’s go…..

[PHASE 1] INFRASTRUCTURE & PRIVACY

[Artifact 1] Offline-First RAG System with Quantized SLMs



Problem we are solving

Lawyers cannot upload case files to ChatGPT (HIPAA violations).

Doctors cannot use AI for patient records.

Privacy regulations block them.

Anyone with sensitive documents is stuck choosing between privacy and productivity.

What you actually learn (Contextual Curriculum)

Module 1 - Quantization fundamentals

- how neural networks store weights (32-bit floating point by default)
- quantization methods (GGUF, GPTQ, AWQ)
- converting models to 4-bit without destroying quality
- memory calculations (how much RAM you actually need)

Module 2 - Running LLMs locally

- llama.cpp architecture and C++ bindings
- cpu inference optimization
- model loading strategies (lazy loading vs full load)
- handling out-of-memory errors gracefully

Module 3 - RAG pipeline basics

- document chunking strategies (semantic vs fixed-size)
- text embedding models (all-MiniLM, BGE, nomic-embed)
- vector databases for local storage (ChromaDB, FAISS)
- similarity search algorithms (cosine similarity, dot product)

Module 4 - Building the retrieval system

- embedding documents locally
- query-time retrieval logic
- context window management (fitting retrieved chunks into prompts)
- re-ranking results for better accuracy

Module 5 - Desktop app development

- streamlit basics for quick UIs
- file upload handling
- real-time streaming responses
- packaging Python apps for distribution

Tech stack

llama.cpp, ChromaDB, Streamlit, Python, Sentence Transformers

Proof of work (what to showcase)

Deploy the desktop app.

Get 10 beta users (lawyers, doctors, anyone with private docs).

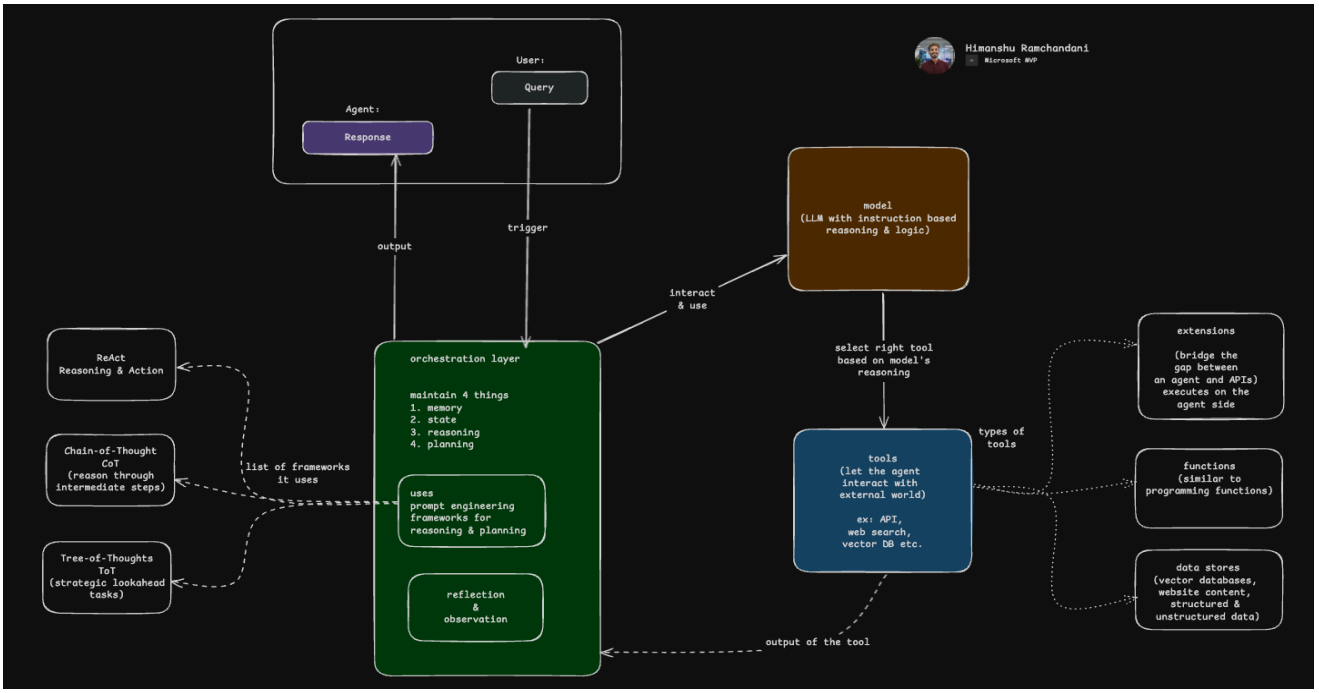
Record a demo video showing:

- upload a sensitive document
- ask complex questions
- get accurate answers
- prove nothing left the device (disconnect wifi mid-conversation)

Write a blog post - I Built ChatGPT for Lawyers That Runs Offline

[PHASE 2] AGENTIC ORCHESTRATION

[Artifact 2] Self-Healing Coding Agent with Stateful Agentic Loops



Problem we are solving

Developers waste hours debugging.

Most AI coding tools generate code but do not fix their own mistakes.

They produce broken code, you fix it manually, and the cycle repeats.

What if an AI could write code, run it, read error logs, and fix its own bugs until tests pass?

What you actually learn (Contextual Curriculum)

Module 1 - Agent fundamentals

- what makes something an agent vs a simple script
- reAct pattern (Reasoning + Acting)
- planning vs reactive agents
- when to use which architecture

Module 2 - State management in agents

- stateful vs stateless loops
- langGraph for building cyclic workflows
- memory persistence across iterations
- tracking previous attempts and failures

Module 3 - Safe code execution

- docker basics (containers, images, volumes)
- docker SDK for Python
- spinning up containers programmatically
- cleaning up resources after execution

Module 4 - Code validation

- abstract Syntax Trees (AST) in Python
- parsing code before running it
- detecting syntax errors statically
- security checks (blocking dangerous operations)

Module 5 - Reflection and self-correction

- prompt engineering for error analysis
- structured outputs from LLMs (JSON mode)
- extracting actionable fixes from error messages
- loop termination conditions (max attempts, success criteria)

Module 6 - Testing and verification

- pytest basics
- writing tests that agents can understand
- automated test execution
- verifying code correctness programmatically

Tech stack

LangGraph, Docker SDK, PyTest, AST, Gemini API, Python.

Proof of work (what to showcase)

Record a video of your agent fixing a real bug. Show:

- give it a broken Python function
- watch it run the code, fail, read the error
- see it modify the code and try again
- watch it succeed after 2-3 attempts

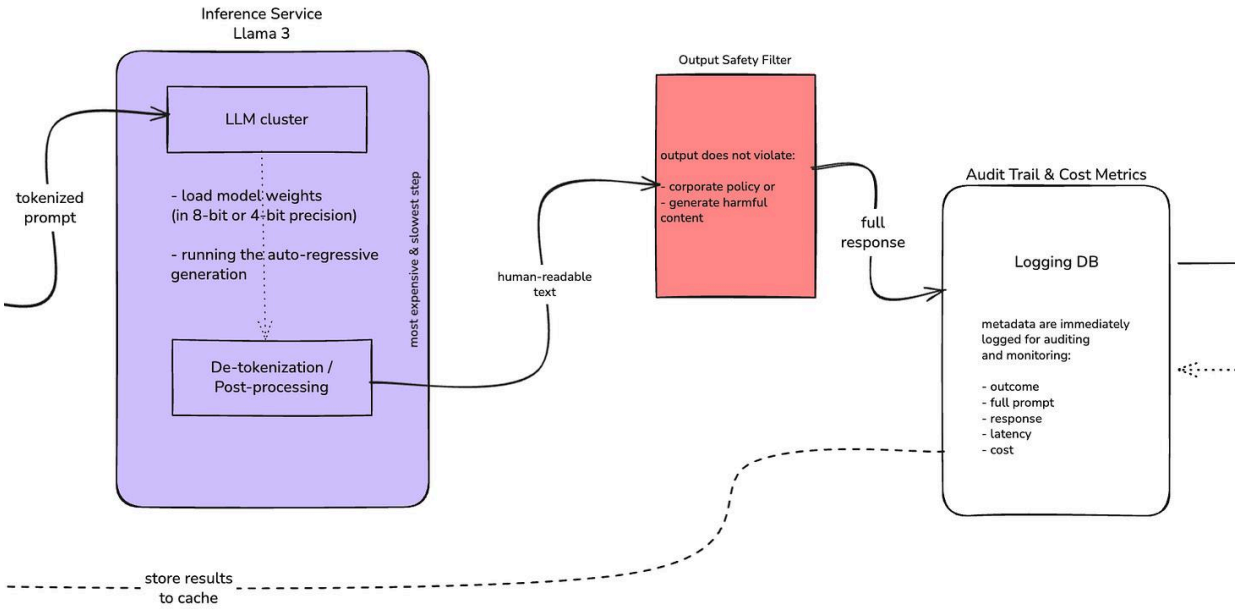
Write a technical blog post breaking down the architecture.

Include the LangGraph diagram, code snippets for the reflection prompt, and examples of bugs it fixed.

Post content on - I built an AI that debugs itself.

[Phase 3] REAL-TIME INTERFACES

[Artifact 3] Real-Time Low-Latency Full-Duplex Audio Streaming Server



Problem we are solving

Voice AI sounds robotic because most implementations are turn-based.

You talk, it waits, then it responds.

Real humans interrupt each other mid-sentence.

Current voice bots cannot handle this.

They feel unnatural and frustrating to use.

What you actually learn (Contextual Curriculum)

Module 1 - Real-time communication fundamentals

- http vs WebSockets (why HTTP fails for real-time)
- websocket protocol basics
- connection lifecycle (connect, message, disconnect)
- handling reconnections and network failures

Module 2 - Audio streaming basics

- audio formats (WAV, MP3, PCM)
- sample rates and bit depth
- chunking audio for streaming (chunk size tradeoffs)
- buffering strategies

Module 3 - Speech-to-text integration

- streaming STT vs batch processing
- deepgram streaming API
- handling partial transcripts

- end-of-speech detection

Module 4 - Voice Activity Detection (VAD)

- silero VAD model
- detecting speech vs silence in audio streams
- detecting interruptions
- calibrating sensitivity thresholds

Module 5 - LLM streaming

- openAI Realtime API
- server-sent events (SSE)
- streaming tokens as they generate
- stopping generation mid-stream when interrupted

Module 6 - Text-to-speech

- elevenLabs streaming API
- latency optimization techniques
- voice cloning basics
- handling pronunciation edge cases

Module 7 - Concurrency and async programming

- python asyncio fundamentals
- managing multiple concurrent streams
- non-blocking I/O
- race conditions and deadlocks

Module 8 - Latency optimization

- measuring end-to-end latency
- profiling bottlenecks
- parallel processing where possible

- caching strategies

Tech stack

FastAPI, WebSockets, Silero VAD, Deepgram, OpenAI Realtime API, ElevenLabs, Python asyncio.

Proof of work (what to showcase)

Deploy it live.

Give friends the URL to call it.

Record the best conversations showing:

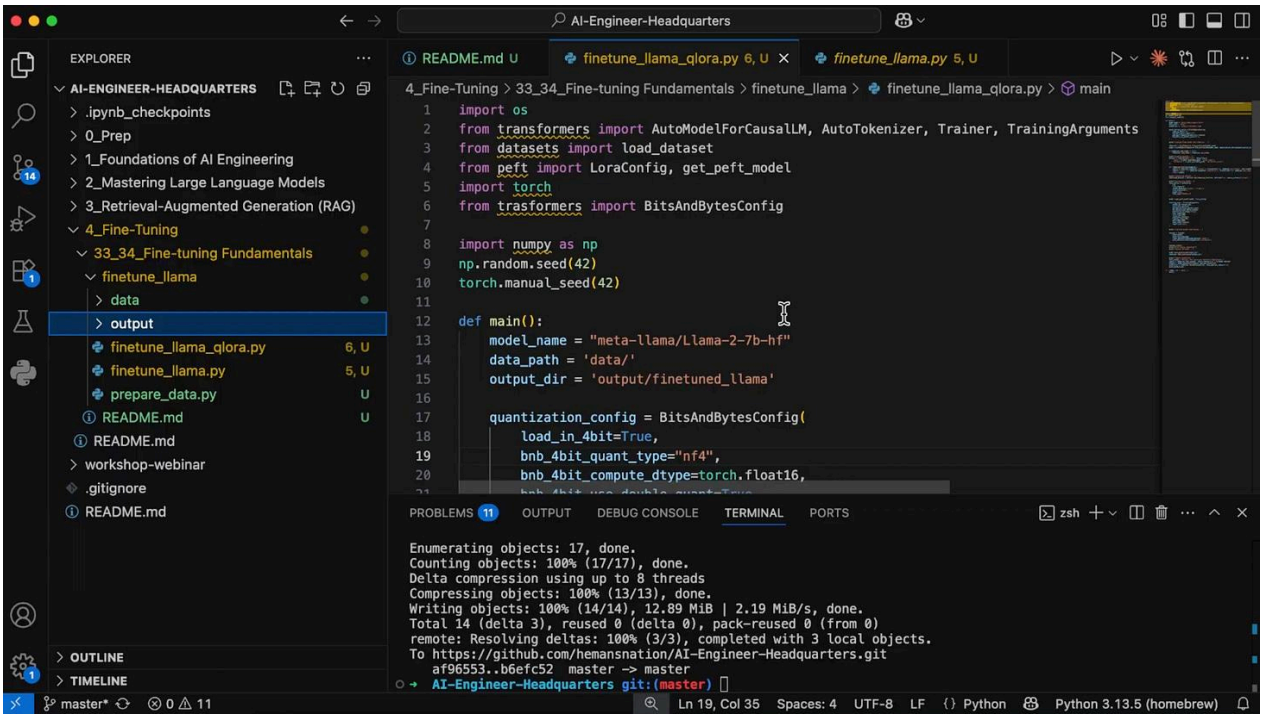
- natural interruptions handled smoothly
- sub-500ms response times
- realistic voice quality
- complex multi-turn dialogue

Write a technical deep-dive - How I Built a Voice AI That Feels Human

Include architecture diagrams showing the WebSocket flow, latency measurements, and code for handling interruptions.

[Phase 4] MODEL SPECIALIZATION

[Artifact 4] Synthetic Data Generation & Fine-Tuning Specialist



Problem we are solving

GPT-4 costs $30 per million tokens.

For high-volume tasks like generating SQL queries, writing product descriptions, or answering support tickets, these costs destroy budgets.

Meanwhile, GPT-4 is overkill for most tasks.

A small 8B model fine-tuned for one specific job can match GPT-4 quality at 1% of the cost.

What you actually learn (Contextual Curriculum)

Module 1 - Synthetic data generation

- prompting strategies for data generation
- few-shot examples for consistent outputs
- quality control (detecting bad synthetic examples)
- scaling generation (batching, parallelization)

Module 2 - Dataset curation

- deduplication techniques
- filtering low-quality examples
- balancing dataset distribution
- train/validation/test splits

Module 3 - Instruction tuning format

- conversation templates
- system prompts vs user prompts

- data formatting for different model families (Llama, Mistral, Phi)
- tokenization and special tokens

Module 4 - Fine-tuning fundamentals

- full fine-tuning vs LoRA
- how LoRA works (low-rank adaptation matrices)
- qLoRA (quantized LoRA for 4-bit training)
- hyperparameter selection (learning rate, batch size, epochs)

Module 5 - Training infrastructure

- gpu selection and cost optimization
- using cloud GPUs (RunPod, Lambda Labs)
- monitoring training (loss curves, gradient norms)
- checkpointing and recovery

Module 6 - Evaluation methods

- traditional metrics (BLEU, ROUGE, exact match)
- llm-as-a-Judge evaluation
- building automated benchmarks
- human evaluation sampling

Module 7 - Deployment

- exporting models to GGUF
- model quantization for inference
- hosting options (Hugging Face, Replicate, local)
- api wrapping with FastAPI

Tech stack

Unsloth, Hugging Face TRL, WandB, Ragas, GPT-4 API, Python.

Proof of work (what to showcase)

Fine-tune a model on a real task (SQL generation, email writing, data extraction).

Run benchmarks comparing it to GPT-4. Create graphs showing:

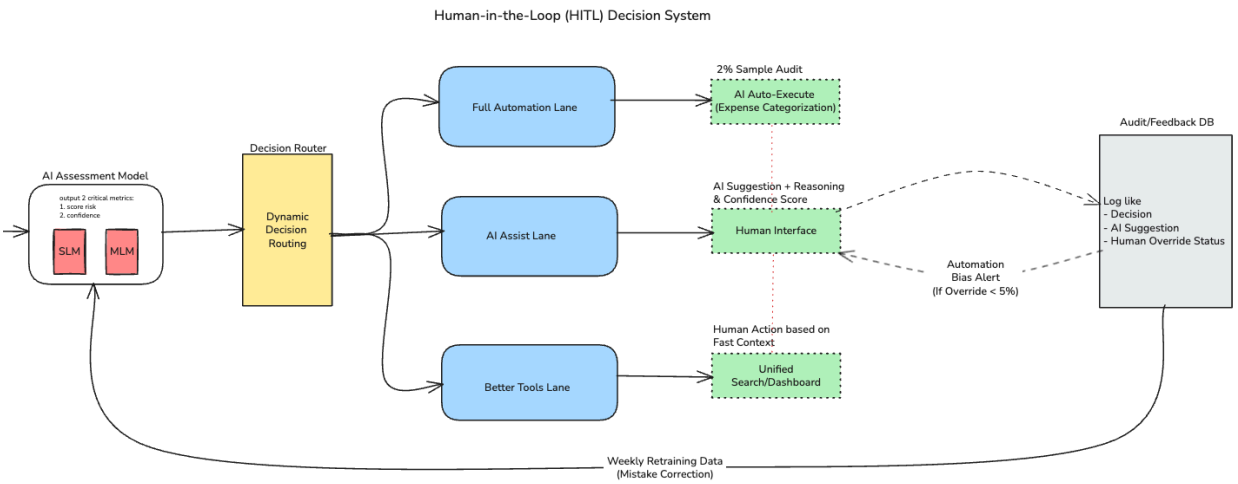
- accuracy comparison
- cost per 1M tokens comparison
- latency comparison

Write a blog post with the results - How I Built a Model That Beats GPT-4 at SQL for 1% of the Cost

Include the training code, dataset generation prompts, and evaluation methodology.

[PHASE 5] ENTERPRISE SCALE

[Artifact 5] Event-Driven Workflow Orchestration & Human-in-the-Loop System



Problem we are solving

Customer support tickets pile up.

Humans cannot respond fast enough.

But fully autonomous AI sends wrong answers and damages customer trust.

Companies need AI that does 80% of the work and asks humans to approve the risky 20%.

This is the human-in-the-loop problem that every enterprise faces.

What you actually learn (Contextual Curriculum)

Module 1 - Event-driven architecture fundamentals

- events vs requests (when to use which)
- webhooks basics
- setting up webhook endpoints with FastAPI
- securing webhooks (signature verification)

Module 2 - Message queues

- why queues matter (decoupling, reliability)
- redis for lightweight queuing
- pub/sub patterns
- handling backpressure

Module 3 - Multi-agent orchestration

- designing agent teams (researcher, writer, reviewer)
- task routing logic
- inter-agent communication
- handoffs and delegation

Module 4 - State persistence

- why agents need databases

- postgreSQL basics
- storing conversation history
- resuming workflows after crashes or restarts

Module 5 - LangGraph for workflows

- building stateful graphs
- conditional edges (if-then routing)
- loops and cycles
- human-in-the-loop nodes (pausing execution)

Module 6 - Integration with external tools

- slack API (reading messages, sending notifications)
- jira API (creating tickets, updating status)
- oauth flows for user authentication
- rate limiting and retries

Module 7 - Monitoring and observability

- logging workflows with LangSmith
- tracing agent decisions
- debugging failures in production
- alerting when agents get stuck

Tech stack

LangGraph, PostgreSQL, Redis, Webhooks, Slack API, FastAPI, Python.

Proof of work (what to showcase)

Deploy it internally at a company or for a side project.

Track metrics:

- time saved per ticket

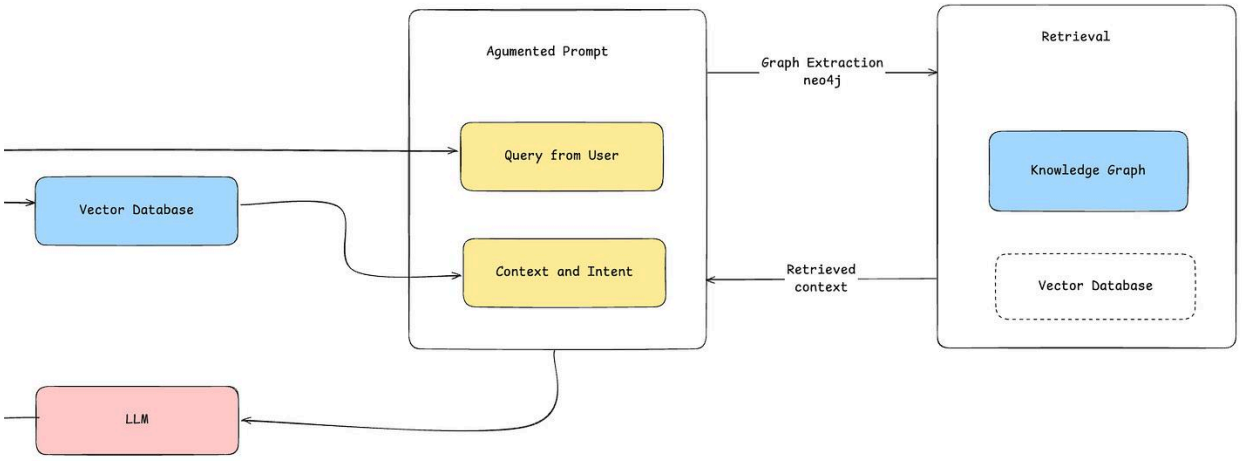
- approval rate (how often humans approve vs reject)
- resolution time (before vs after)

Write a case study - How I Built an AI Support Agent That Needs Human Approval

Include architecture diagrams showing the event flow, screenshots of Slack approvals, and code for the state machine.

[PHASE 6] ADVANCED MEMORY

[Artifact 6] Hybrid Retrieval Memory Systems (Vector + Graph)



Problem we are solving

AI assistants forget context.

They do not remember who you talked about last week, what projects you are working on, or relationships between people and ideas.

Vector search finds similar content but misses connections.

Graph search finds relationships but misses semantic meaning.

Most engineers only know one.

You need both.

What you actually learn (Contextual Curriculum)

Module 1 - Vector databases deep dive

- embeddings review (how text becomes vectors)
- HNSW algorithm (approximate nearest neighbors)
- index types and tradeoffs
- metadata filtering

Module 2 - Graph databases fundamentals

- nodes, edges, and properties
- neo4j Cypher query language
- modeling relationships (entities, events, concepts)
- traversal algorithms (shortest path, PageRank)

Module 3 - Knowledge graph construction

- entity extraction from text (NER)
- relationship extraction
- coreference resolution (knowing “he” refers to “John”)
- building graphs programmatically

Module 4 - Hybrid search strategies

- when to use vector vs graph vs keyword search
- combining results from multiple systems
- scoring and ranking hybrid results
- query routing logic

Module 5 - Memory compression

- recursive summarization
- forgetting strategies (what to keep vs discard)
- context window management
- long-term vs short-term memory

Module 6 - Style transfer and personalization

- few-shot prompting for writing style
- extracting stylistic features from text
- voice and tone consistency
- testing personalization quality

Module 7 - Privacy and encryption

- end-to-end encryption basics
- storing sensitive data securely
- key management
- compliance considerations (GDPR)

Tech stack

Neo4j, NetworkX, LangChain, Sentence Transformers, Cryptography libraries, Python

Proof of work (what to showcase)

Use the system for two weeks.

Document:

- examples of it recalling past conversations accurately
- emails it wrote in your style
- relationships it remembered (people, projects, context)

Join the Accelerator (Cohort 1)

6 ARTIFACTS IN 8 WEEKS **AI Engineer Accelerator**

This is not a course. It is a simulation of a high-performance engineering team. We start in **7 Days**.

If you are ready to stop building wrappers and start architecting systems...

[SECURE YOUR SPOT]

https://www.masterdexter.io/ai-engineer-hq

⚠️ **Note:** I offer a Regional Purchasing Parity Grant (up to 60% off) for builders in India, LATAM, and Africa. If this applies to you, click the link and DM me.

About Me

I’m Himanshu Ramchandani, from India.

Microsoft MVP

I am an AI Engineer & Consultant with close to a decade of experience.

I worked on over 100 Data & AI projects in Energy, Healthcare, Law Enforcement & Defense.

I am the Founder of an AI engineering & Consulting company - Dextar.

I focus on action-oriented AI leadership & engineering implementation drills.

I provide AI Engineering & Leadership training to teams through AI Engineer HQ and The Elite [AI Leadership Accelerator]

In the last decade, I have never stopped sharing my knowledge and have helped over 10000 leaders, professionals, and students.

https://www.masterdexter.io/ai-engineer-hq